

16-bit 3-Stage Pipelined ALU — Complete Project

Stages: Decode → Execute → Writeback

Instruction format: [15:12] opcode, [11:9] rs1, [8:6] rs2, [5:3] rd, [2:0] reserved.

Implemented: 8-register × 16-bit register file, 16-bit ALU, ID/EX and EX/WB pipeline registers, Z/C/V/N flags, RAW forwarding, writeback, testbench and VCD waveform generation.

1. Stage-wise architecture

```
Input Instruction
|
v
+-----+
| STAGE 1: DECODE |
| opcode |
| rs1 / rs2 / rd |
| operand A / B |
+-----+
|
ID/EX
|
v
+-----+
| STAGE 2: EXECUTE |
| 16-bit ALU |
| result + Z/C/V/N |
+-----+
|
EX/WB
|
v
+-----+
| STAGE 3: WB |
| register write |
+-----+
```

2. What is completed

Stage 1 captures opcode, source operands and destination register. Stage 2 performs arithmetic/logic operations and generates status flags. Stage 3 commits the result into the register file.

3. RAW hazard handling

The complete version includes simple forwarding. If the instruction in Execute is producing a value needed by the instruction being decoded, the ALU result is forwarded directly. EX/WB forwarding is also available for the following dependency.

4. Demonstration program

```
Initial registers:
R2 = 10
R3 = 20

1) ADD R1,R2,R3 -> R1 = 30
2) SUB R4,R1,R2 -> R4 = 20 (RAW forwarding)
3) AND R5,R1,R3 -> R5 = 20
4) OR R6,R1,R2 -> R6 = 30
5) XOR R7,R1,R2 -> R7 = 20
```

5. Expected final result

```
R1 = 30
R4 = 20
R5 = 20
R6 = 30
R7 = 20
```

PASS: Complete 3-stage pipeline verified.

6. EDA Playground setup

Put `pipelined_alu_3stage_design.v` in the Design window and `pipelined_alu_3stage_testbench.v` in the Testbench window. Select Icarus Verilog. Set the top module to `tb_pipelined_alu_3stage`. Run the simulation. The testbench generates `dump.vcd`, so EPWave can be opened after the run.

7. What to tell the professor

"I have implemented a 16-bit three-stage pipeline. Stage 1 decodes the instruction and prepares operands, Stage 2 executes the ALU operation and generates flags, and Stage 3 writes the result back to the register file. I also added basic RAW forwarding so dependent instructions can execute back-to-back. The testbench verifies ADD, SUB, AND, OR and XOR and checks the final register values."

8. Next development after this baseline

For a stronger final project, the next additions can be a proper instruction memory/program counter, more complete hazard/forwarding logic, stalls for cases forwarding cannot resolve, reset/flush behavior, branch support if required, and larger automated verification.